UTEC — Maestría en Ciencia de Datos e Inteligencia Artificial · Estructuras de Datos y Algoritmos 2026-2 · Prof. Heider Sánchez

EcoStream: estructuras de datos avanzadas orquestadas en un pipeline de data streams para comercio electrónico

Grupo 3 — Integrante 1 · Integrante 2 · Integrante 3 · Integrante 4

Repositorio: `proyecto-eda-grupo3` · Escenario B (comercio electrónico)

Resumen. Diseñamos, implementamos y evaluamos **EcoStream**, un pipeline de procesamiento de streams de comercio electrónico de diez etapas en el que cada etapa usa la estructura de datos más adecuada y su salida alimenta a otra de forma verificable: cola FIFO, Merge Sort y búsqueda binaria, heap de prioridades, tabla hash, LRU y matriz dispersa, B+Tree paginado en disco, KD-Tree, Bloom Filter y Count-Min Sketch para fraude, MinHash+LSH y HNSW para similitud, grafo de co-compra con PageRank y Louvain, y un servicio con Trie, Top-K y panel. Todas las estructuras son implementaciones propias en Python, validadas con 14 pruebas contra referencias. Con datasets de 10^3 a 10^7 elementos mostramos que las complejidades teóricas se cumplen en log-log; que el Bloom Filter alcanza su FPR teórica (1.008 % frente a 1.004 %) con 52× menos memoria que un `set`; que MinHash+LSH responde ∼9 000× más rápido que la fuerza bruta con 10^7 conjuntos sin perder vecinos con $J \geq 0.5$; que HNSW es 54× más rápido que el KNN exacto con recall@10 = 0.989 en 10^6 vectores; y que Louvain es más estable que Label Propagation y logra mayor modularidad en el grafo real de co-compra (0.755 frente a 0.630). El pipeline procesa 10M eventos de punta a punta.

1 Contexto y motivación

Una tienda en línea recibe un flujo continuo de vistas, búsquedas, carritos, pagos, compras, devoluciones y reseñas. Sobre ese flujo el negocio necesita, con baja latencia: atender primero los pagos y las órdenes urgentes; consultar productos por clave y por rango; detectar medios de pago o dispositivos señalados y ráfagas de intentos (*card testing*) *antes* de aprobar una orden; recomendar productos relacionados y complementarios; e identificar comunidades de co-compra y productos influyentes para *cross-selling*. Ninguna estructura resuelve todo: cada requisito tiene una estructura óptima distinta y el valor del sistema está en **orquestarlas** de modo que la salida de cada etapa sea la entrada real de otra. El escenario elegido es el **B (comercio electrónico)**; el grafo de dominio tiene a los productos como nodos y una arista ponderada entre productos comprados en la misma orden.

Datos. Como base estructural se adoptó el esquema del dataset público *eCommerce behavior data from multi category store* (REES46, Kaggle: `event_time, event_type, product_id, category_code, brand, price, user_id, user_session`); el repositorio incluye un cargador (`rees46_loader.py`) que alimenta el mismo pipeline con el CSV real. Siguiendo la recomendación del enunciado, el componente de *stream* se genera sintéticamente y de forma vectorizada (10M eventos en ∼20 s) con propiedades controladas por semilla: catálogo de 20 000 productos en 40 categorías con popularidad Zipf, afinidad usuario→categoría (comunidades reales) y categorías complementarias plantadas; órdenes de 1–6 ítems con envío express/estándar/programado; 2 % de re-entregas duplicadas, 0.5 % de registros sucios (tipo en mayúsculas, precio con coma decimal, faltante o negativo, categoría en mayúsculas); 500 tokens de pago comprometidos y ráfagas de 25–80 intentos de pago en ≤3 minutos, la mitad con tokens nuevos que solo una estimación de frecuencia puede detectar.

2 Arquitectura del pipeline

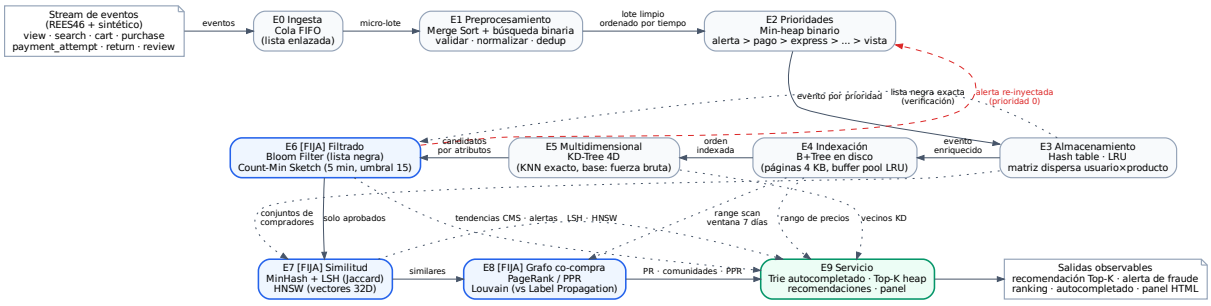


Figura 1: Arquitectura de 10 etapas. Flechas sólidas: flujo principal de cada evento; punteadas: salidas de una etapa consumidas por otra (lista negra exacta, conjuntos de compradores, *range scan* temporal, vecinos, tendencias); discontinua roja: realimentación de alertas de fraude a la cola de prioridades con prioridad 0.

El procesamiento es por **micro-lotes** (como Spark Structured Streaming): E0–E6 actúan evento a evento sobre cada lote y E7–E8, cuyo costo es global, se recalculan cada 20 lotes con los datos que ya pasaron el filtro. La integración es verificable: el script `scripts/demo.py` marca eventos concretos y registra lo que cada etapa hizo con ellos (Sección 6.1). Las etapas 0–5 se resumen en la Tabla 1; las etapas FIJA 6–8 se documentan en profundidad en la Sección 3.

Cuadro 1: Etapas 0–5 y 9: función, estructura (implementación propia), complejidad y justificación frente a alternativas.

Etapa	Estructura	Complejidad	Función en el pipeline y justificación
E0 Ingesta	Cola FIFO con lista enlazada y capacidad máxima	enqueue/dequeue $O(1)$; lote $O(b)$	Buffer del stream con *backpressure*. `list.pop(0)` es $O(n)$; un arreglo circular exige capacidad fija. Entrega micro-lotes a E1.
E1 Preproceso	Merge Sort estable (runs de 16 con inserción) + búsqueda binaria	$O(b \log b)$ por lote; $O(\log W)$ por id	Valida y normaliza; categoría validada por búsqueda binaria; deduplica intra-lote (ordenar por `event_id` + adyacentes) y entre lotes (búsqueda binaria en ventana ordenada de W=300K ids). Estable y sin peor caso $O(n^2)$ (Quicksort). Reordena por tiempo.
E2 Prioridades	Min-heap binario en arreglo, clave (prioridad, tiempo)	push/pop $O(\log n)$	9 clases: alerta > pago > express/premium > estándar > programada > devolución > carrito > reseña > vista. Lista ordenada: inserción $O(n)$. El desempate por tiempo evita inanición dentro de cada clase.
E3 Almacén	Tabla hash (sondeo lineal, carga 0.7), LRU (hash + lista doble), matriz dispersa DOK→CSR	$O(1)$ esperado; LRU $O(1)$; $O(nnz)$ memoria	Producto, usuario, orden, pago y lista negra exacta. LRU para productos y resultados calientes (Zipf). Matriz usuario×producto: densa costaría 8 GB a $10^5 \times 10^4$. Enriquece el evento y alimenta E6, E7 y E9.
E4 Índice disco	B+Tree con páginas de 4 KB en archivo + *buffer pool* LRU (write-back)	$O(\log_B n)$ páginas; rango $O(\log_B n + k/B)$	Índice temporal de órdenes (E8 lee la ventana de 7 días con *range scan*) y de precios (E9). Fan-out $B \approx 200$: altura 4 con 10^7 claves vs. ∼24 de un AVL.
E5 Multidim.	KD-Tree 4D (mediana, dimensión de mayor dispersión, hojas de 32) + KNN fuerza bruta como base	construcción $O(n \log n)$; consulta $\sim O(\log n)$ en d bajo	Productos con atributos parecidos (precio, rating, popularidad, descuento) para cada vista; los productos son puntos (no rectángulos), por eso KD-Tree y no R-Tree. Resultados cacheados en LRU.
E9 Servicio	Trie con top-K cacheado por nodo, Top-K con min-heap de tamaño K	autocompletar $O(\|p\| + K)$; Top-K $O(n \log K)$	Autocompletado ponderado por ventas×PageRank; recomendación por fusión de PPR (E8), LSH y HNSW (E7) y KD (E5); tendencias del CMS; panel HTML.

3 Investigación avanzada: etapas 6–8

Para cada familia se resume fundamento, funcionamiento interno, complejidad, casos de uso, límites y *trade-offs*, el paper original, implementaciones de referencia (contra las que se validó la nuestra en `tests/`) y la evidencia empírica frente a la alternativa exacta o más simple (Sección 6). Todas las estructuras están implementadas por el grupo en `src/ecostream/`.

3.1 Filtrado probabilístico: Bloom Filter + Count-Min Sketch (E6)

Bloom Filter (Bloom, 1970). *Fundamento:* representar un conjunto S con un arreglo de m bits y k funciones hash; aceptar falsos positivos a cambio de memoria sublineal en el tamaño de las claves. *Funcionamiento:* `add`(x) enciende los bits $g_i(x)$, $i = 1..k$; `contains`(x) responde *posiblemente presente* si todos están encendidos y *definitivamente ausente* si alguno está apagado (sin falsos negativos). Usamos *double hashing* $g_i(x) = h_1(x) + i\,h_2(x) \bmod m$ (Kirsch y Mitzenmacher, 2006), que conserva la FPR asintótica con solo dos hashes de 64 bits (splitmix64 sobre BLAKE2b). *Complejidad:* $O(k)$ por operación, independiente de n; memoria $m = -n \ln p / (\ln 2)^2$ bits (≈9.6 bits/elemento para $p = 1\,\%$) con $k = (m/n) \ln 2$; FPR teórica $(1 - e^{-kn/m})^k$. *Casos de uso:* listas negras, caché de "un solo acceso", evitar lecturas a disco en LSM-trees (LevelDB, Cassandra), *joins* distribuidos. *Límites:* no admite borrado (*Counting Bloom* o *Cuckoo filter*, Fan et al., 2014), exige estimar n por adelantado (si se excede, la FPR crece rápidamente; *Scalable Bloom Filters*) y un positivo nunca es prueba. *Referencias de implementación:* Guava `BloomFilter`, RedisBloom, `pybloom-live`. *Decisión en el pipeline:* el filtro contiene tokens de pago y dispositivos señalados ($n = 50\,000$, $p = 0.1\,\%$, 88 KB). Un positivo **no bloquea**: se verifica en la tabla hash exacta de E3 (decisión crítica); solo si se confirma se bloquea, y si no, se contabiliza como falso positivo y la orden continúa. Así el Bloom ahorra la consulta exacta en el ∼99.9 % de pagos legítimos sin introducir bloqueos erróneos.

Count-Min Sketch (Cormode y Muthukrishnan, 2005). *Fundamento:* resumen lineal de un *stream* que estima frecuencias con error aditivo acotado en memoria independiente del número de claves. *Funcionamiento:* matriz $d \times w$ de contadores y d hashes por pares independientes; `update`(x, c) suma c en $C[i, h_i(x)]$ para cada fila y $\hat{f}(x) = \min_i C[i, h_i(x)]$. *Garantía:* con $w = \lceil e/\varepsilon \rceil$ y $d = \lceil \ln(1/\delta) \rceil$, $f(x) \le \hat{f}(x) \le f(x) + \varepsilon N$ con probabilidad $\ge 1 - \delta$ (nunca subestima). *Complejidad:* $O(d)$ por operación; memoria $O(wd)$. *Casos de uso: heavy hitters*, tendencias, detección de DDoS y de velocidad de transacciones, conteos aproximados en bases columnares. *Límites:* el error es relativo a N (malo para claves raras), sesgo positivo (Count Sketch de Charikar et al. es insesgado; *conservative update* reduce la sobreestimación). *Referencias:* RedisBloom (CMS), Apache DataSketches, `stream-lib`. *Decisión en el pipeline (clave, ventana y umbral exigidos):* **clave** = `payment_token` y `device_id` de cada intento de pago; **ventana** deslizante de 5 minutos implementada como anillo de 5 sub-sketches de 60 s ($\varepsilon = 5 \cdot 10^{-4}$, $\delta = 10^{-3}$); **umbral** = más de 15 intentos en la ventana (un cliente legítimo hace 1–3). Al superarlo se genera

una alerta que (i) se re-inyecta en el heap con prioridad 0, (ii) agrega token y dispositivo al Bloom y a la lista negra exacta y (iii) se muestra en el panel. Un segundo CMS (ventana de 1 h) cuenta compras por producto y alimenta las tendencias de E9.

3.2 Similitud de conjuntos: MinHash + LSH (E7)

Fundamento (Broder, 1997; Indyk y Motwani, 1998): para una permutación aleatoria π del universo, $\Pr[\min\pi(A) = \min\pi(B)] = J(A,B)$. Con k permutaciones (aproximadas por hashes universales $h_i(x) = (a_i x + b_i) \bmod (2^{31}-1)$) la fracción de coincidencias estima J con error estándar $\sqrt{J(1-J)/k}$. *LSH por bandas:* la firma se divide en b bandas de r filas y dos conjuntos son candidatos si coinciden en una banda completa: $\Pr[\text{candidato}] = 1-(1-J^r)^b$, una curva S con umbral $\approx (1/b)^{1/r}$. *Complejidad:* firmas $O(k|A|)$ por conjunto; índice $O(nb)$; consulta $O(b\log n + |C|\,k)$ frente a $O(n\,|\bar{A}|)$ de la fuerza bruta. Nuestra implementación guarda por banda un arreglo ordenado de claves de 64 bits y responde con búsqueda binaria (`searchsorted`), y construye el índice por bloques sin materializar la matriz de firmas (10M conjuntos en ∼2.6 GB). *Casos de uso:* casi-duplicados web, deduplicación de catálogos, *item-item* colaborativo, plagio. *Límites:* solo Jaccard; elegir (b,r) fija el compromiso recall/candidatos; conjuntos muy pequeños tienen estimaciones ruidosas. *Referencias:* `datasketch` (Python), Spark MLlib `MinHashLSH`; Leskovec et al. (2020, cap. 3). *Selección frente a SimHash* (Charikar, 2002): la señal de comportamiento es un conjunto binario (usuarios que interactuaron con el producto), cuya similitud natural es Jaccard; SimHash aproxima coseno sobre vectores ponderados. *Parámetros:* $k = 64$, $b = 32$, $r = 2$ (umbral 0.18), elegidos con el experimento de la Fig. 4: en este dominio la Jaccard producto–producto rara vez supera 0.3, y la configuración típica 16×4 (umbral 0.5) recuperaba menos del 10 % de los vecinos reales del pipeline.

3.3 ANN vectorial: HNSW (E7)

Fundamento (Malkov y Yashunin, 2020): grafos de mundo pequeño navegables más una jerarquía tipo *skip list*. *Funcionamiento:* cada vector recibe un nivel $l = \lfloor -\ln U \cdot m_L \rfloor$, $m_L = 1/\ln M$; las capas altas son "autopistas" con pocos nodos y la capa 0 contiene a todos. La inserción baja voraz hasta l y en cada capa ejecuta `search_layer` con ef_c candidatos, elige M vecinos con la **heurística de diversidad** (un candidato se acepta si está más cerca del nuevo nodo que de los ya elegidos; preserva puentes entre clústeres) y poda los vecinos que exceden M_{max}. La consulta repite el descenso y explora la capa 0 con $ef \geq k$. *Complejidad:* construcción $O(n\log n)$, consulta $\sim O(\log n)$ empírica, memoria $O(n(d+M))$. *Casos de uso:* búsqueda semántica, recomendación por *embeddings*, RAG. *Límites:* aproximado (recall < 1 si ef es bajo), memoria del grafo, borrado costoso, construcción cara en Python. *Referencias:* `hnswlib`, FAISS `IndexHNSWFlat`; en ANN-Benchmarks (Aumüller et al., 2020) los métodos basados en HNSW dominan el frente recall–QPS. *Selección frente a IVF y M-Tree/VP-Tree:* el catálogo crece con inserciones incrementales (HNSW no requiere reentrenar centroides como IVF, Jégou et al., 2011) y el espacio de *embeddings* normalizados es euclidiano (equivalente a coseno), por lo que no se necesita un índice métrico general. El KD-Tree de E5 se mantiene para 4 dimensiones: en 32D visita casi todas las hojas (Fig. 4), evidencia directa de la maldición de la dimensionalidad.

3.4 Ranking de influencia: PageRank y Personalized PageRank (E8)

Fundamento (Brin y Page, 1998): la importancia de un nodo es la probabilidad estacionaria de un caminante aleatorio que sigue aristas con probabilidad $d = 0.85$ y se teletransporta con $1-d$. En el grafo de co-compra ponderado, $P_{ij} = w_{ij}/s_i$ y $r_{t+1} = d\,P^\top r_t + (d\,r_{\text{colg}} + 1 - d)\,v$. *Funcionamiento:* iteración de potencias hasta $\|r_{t+1} - r_t\|_1 < 10^{-9}$; cada iteración es un producto matriz dispersa–vector $O(m)$ (implementado con `np.bincount`); converge geométricamente con razón d. *Personalized PageRank* (Haveliwala, 2002) usa v concentrado en las semillas (carrito y compras del usuario) y mide afinidad con ellas: es la señal principal del recomendador. *Límites:* en grafos no dirigidos PageRank está muy correlacionado con el grado; es costoso recalcular PPR por usuario (se cachea en LRU). *Referencias:* `networkx.pagerank` (nuestra implementación coincide con un error $< 10^{-6}$ en `tests/`), igraph, Spark GraphX. *Alternativa simple:* ranking por grado ponderado.

3.5 Comunidades: Louvain frente a Label Propagation (E8)

Fundamento: maximizar la modularidad $Q = \sum_c \left[L_c/m - (S_c/2m)^2\right]$ (Newman, 2006). **Louvain** (Blondel et al., 2008), el algoritmo elegido, alterna (1) movimientos locales: cada nodo pasa a la comunidad vecina con mayor ganancia $\Delta Q \propto k_{i,c} - S_c k_i/2m$ hasta que no hay mejora, y (2) agregación de cada comunidad en un super-nodo; se repite por niveles. Complejidad empírica $\sim O(m\log n)$, memoria $O(n+m)$. **Label Propagation** (Raghavan et al., 2007), la alternativa más simple: cada nodo adopta la etiqueta de mayor peso entre sus vecinos, en orden aleatorio, hasta que no hay cambios; $O(m)$ por iteración, sin función objetivo y sensible al orden. Girvan–Newman se descartó por su costo $O(m^2 n)$. *Límites:* límite de resolución de la modularidad (Fortunato y Barthélemy, 2007) y comunidades internamente desconectadas en Louvain, que Leiden (Traag et al., 2019) corrige; LPA puede colapsar en una comunidad gigante y varía entre ejecuciones. *Referencias:* `networkx` (`louvain_communities`), `python-louvain`, igraph `community_multilevel`, `leidenalg`; nuestra modularidad coincide con la de networkx y nuestro Louvain alcanza su Q en `tests/`.

4 Diseño e integración

Recorrido de un evento. (1) E0 encola el evento en la FIFO; el consumidor extrae micro-lotes de tamaño variable (0.55–1.4×5 000, para simular picos). (2) E1 normaliza, rechaza precios negativos o tipos desconocidos, marca precios faltantes y elimina duplicados (intra-lote por Merge Sort, entre lotes por búsqueda binaria). (3) E2 inserta el lote en el heap;

por *tick* se atienden a lo más 5 000 eventos (capacidad de servicio), así que en los picos esperan las vistas y no los pagos. (4) E3 enriquece con el registro del producto (vía LRU), completa el precio faltante, actualiza el perfil, la matriz dispersa y la orden. (5) E4 inserta la orden en el B+Tree temporal. (6) E5 obtiene vecinos por atributos para vistas y carritos. (7) E6 decide cada pago: Bloom→verificación exacta y CMS→umbral; las alertas vuelven a E2 con prioridad 0 y el estado (aprobado/bloqueado) se propaga a la orden. (8) Cada 20 lotes, E7 reconstruye MinHash+LSH con los conjuntos de usuarios por producto (solo compras aprobadas) y E8 lee del B+Tree las órdenes **aprobadas** de los últimos 7 días, construye el grafo de co-compra y recalcula PageRank y Louvain. (9) E9 actualiza los pesos del Trie (ventas × PageRank) y responde.

Puntuación de recomendaciones (E9). Para un usuario con semillas S (carrito, últimas compras y vistas), cada candidato p recibe $0.45\,\widehat{\mathrm{PPR}}_S(p) + 0.30\,\hat{J}_{\mathrm{LSH}}(p) + 0.15\,\cos_{\mathrm{HNSW}}(p) + 0.03\,[p \in \mathrm{KD}] + 0.07\,\widehat{\mathrm{PR}}(p)$ (cada término normalizado a $[0,1]$), se excluyen productos ya comprados y se seleccionan los K mejores con un min-heap de tamaño K. Sin historial (arranque en frío) se devuelven los productos de mayor PageRank. Cada recomendación lleva su explicación (qué fuentes la generaron).

Decisiones de implementación. (i) Todas las estructuras son propias; NumPy se usa para vectorizar núcleos numéricos (hashes del Bloom/CMS por lotes, distancias, `bincount` de PageRank), y existen versiones escalares (evento a evento) del Bloom y del CMS que el pipeline usa en línea. (ii) El B+Tree es un índice en disco real: los nodos se serializan en páginas de 4 KB de un archivo y se leen/escriben a través del *buffer pool* LRU de E3, lo que permite medir lecturas de página. (iii) Las etapas globales (E7, E8) son incrementales por micro-lotes y no por evento, ya que Louvain o la reconstrucción del LSH por evento serían $O(m)$ por evento. (iv) Corrección verificada con 14 pruebas (`tests/test_structures.py`) contra `dict`, `heapq`, `bisect`, fuerza bruta y `networkx`.

5 Metodología experimental

Plataforma. Intel Xeon @ 2.30 GHz, 2 vCPU, 7.8 GB de RAM, Linux 6.18, Python 3.13.16, NumPy 2.5.3. **Semilla** global 42 (cada tamaño usa $42+n$ para que los datos sean distintos pero reproducibles). Los experimentos se ejecutaron en dos colas paralelas, una por núcleo, y los de 10^7 de forma secuencial (`experiments/run_all.sh`).

Tamaños. Para cada estructura base y cada etapa FIJA se generaron datasets de 10^3, 10^4, 10^5, 10^6 y 10^7 elementos (operaciones, claves, conjuntos, vectores, aristas o eventos según la estructura). **Repeticiones:** una corrida de calentamiento estima t y luego $\mathrm{reps} = \min(100, \max(r_{\min}, \lfloor 60\,\mathrm{s}/t \rfloor))$, con $r_{\min} = 3$ hasta 10^6 y entre 1 y 3 en 10^7; se reportan media y desviación estándar. Con ello 10^3–10^4 (y casi todo 10^5) alcanzan 100 repeticiones; en 10^6–10^7 se usan menos porque una sola corrida de una estructura en Python puro dura de segundos a minutos y el presupuesto total de cómputo es de 2 vCPU. Cada fila de `experiments/results/*.csv` reporta su número real de repeticiones. **Memoria:** corrida aparte con `tracemalloc` (memoria retenida por la estructura y pico durante la operación; NumPy registra sus *buffers*); en 10^7, donde `tracemalloc` multiplica el tiempo 3–5×, se usa la herramienta equivalente: RSS de `psutil` en un proceso hijo (actual) y `ru_maxrss` (pico). **Escalabilidad:** se grafica en log-log y se ajusta $t = a n^b$ por mínimos cuadrados; la pendiente b se contrasta con la complejidad esperada (1 para lineal, ≈1.05–1.1 para $n \log n$ en este rango, 2 para cuadrática, ≈0 para logarítmica). **Calidad:** FPR empírica sobre n claves ausentes; error del CMS contra un `Counter` exacto; recall@10 de LSH y HNSW contra fuerza bruta; modularidad, número/tamaño de comunidades, estabilidad (NMI media entre pares de semillas) y NMI contra las comunidades plantadas.

6 Resultados

6.1 Demostración de punta a punta

Con 300 000 eventos (`scripts/demo.py`, salida completa en `docs/demo_output.txt`) el pipeline procesa el stream a ∼12 000 eventos/s; con 1M eventos, a 7 600 eventos/s, y con 10M, a 7 674 eventos/s (Tabla 2). Ejemplos de recorridos registrados por el trazador: (i) una **compra express** entra en la FIFO, E1 la valida y verifica por búsqueda binaria que no es duplicada, E2 la inserta con prioridad 2 y sale del heap sin esperar, E3 la enriquece (producto, precio), E4 indexa su orden en el B+Tree, E6 la aprueba, E7 la usa en los conjuntos de compradores, E8 la incorpora al grafo de la ventana y E9 devuelve un Top-5 para el usuario; (ii) un **intento de pago** de una ráfaga supera el umbral del CMS, se bloquea y la alerta re-inyectada se atiende con prioridad 0; (iii) un registro con precio negativo se rechaza en E1 y una re-entrega duplicada se elimina en E1. En ningún tamaño los pagos, alertas o compras esperaron en el heap (0 *ticks*); solo las vistas/búsquedas esperaron en los picos (0.47–0.66 *ticks* en promedio). Los positivos del Bloom se verificaron en la tabla hash antes de bloquear: 0 bloqueos erróneos.

Cuadro 2: Pipeline completo (20 000 productos, micro-lote 5 000; 10 000 en 10M). Costo por etapa en µs por elemento que la etapa procesa.

Eventos	Procesados	s	ev/s	RSS GB	Órd. aprob./bloq.	Alertas	E0	E1	E2	E3	E4	E5	E6
10^4	9 996	1	10 333	0.1	348 / 2	3	10	4	5	5	4	56	91
10^5	99 901	9	11 551	0.2	3 480 / 17	3	5	5	5	5	4	37	99
10^6	999 061	132	7 597	1.1	34 808 / 146	20	7	15	7	7	5	37	78
10^7	9 990 165	1302	7 674	4.5	348 074 / 1405	200	15	15	11	10	5	38	63

La etapa más costosa por elemento es E7 (reconstrucción del LSH y, sobre todo, la construcción del HNSW de 20 000 productos, 40 s fijos de inicialización), seguida por E6 (Bloom + dos CMS escalares por pago, 63–99 µs) y E5 (KD-Tree, ∼37 µs, mitigado por un LRU con 69 % de aciertos). E0–E4 cuestan 4–15 µs por evento.

6.2 Estructuras base (E0–E5, E9)

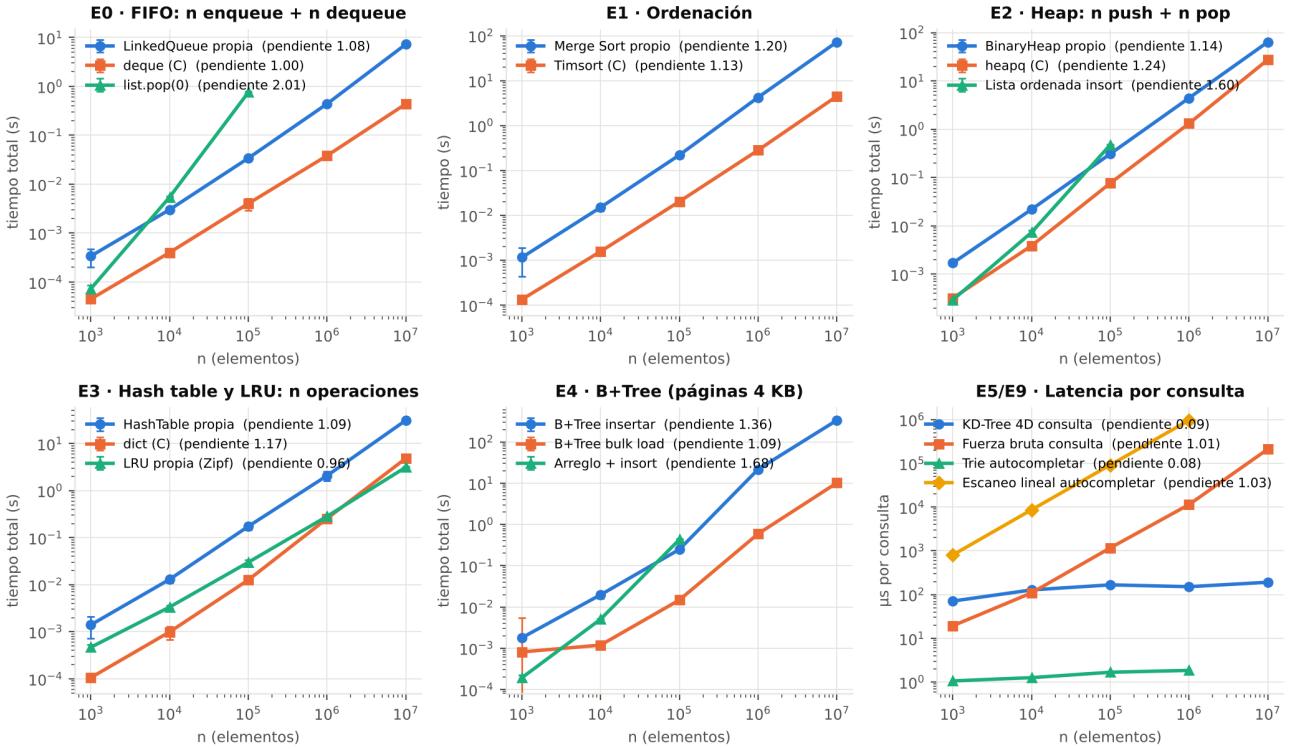


Figura 2: Escalabilidad log-log (media ± desviación estándar) de las estructuras base frente a referencias en C y alternativas ingenuas; en la leyenda, la pendiente ajustada b de $t \propto n^b$ entre 10^3 y el mayor tamaño medido (10^7 salvo las alternativas cuadráticas y el Trie).

Cuadro 3: Pendiente ajustada b ($t \propto n^b$, de 10^3 al mayor tamaño medido) frente a la esperada; para latencias por consulta, $b \approx 0$ indica costo sublineal.

Estructura / operación	b	Esperada	Estructura / operación	b	Esperada
LinkedQueue propia	1.08	1	Escaneo lineal autocompletar	1.03	1
deque (C)	1.00	1	Bloom insertar	1.00	1
list.pop(0)	2.01	2	set insertar	1.13	1
Merge Sort propio	1.21	≈1.05-1.1	Bloom consultar 2n	1.01	1
Timsort (C)	1.13	≈1.05-1.1	set consultar 2n	1.15	1
BinaryHeap propio	1.15	≈1.05-1.1	MinHash+LSH consulta	0.06	≈0
heapq (C)	1.24	≈1.05-1.1	Fuerza bruta Jaccard	1.12	1
Lista ordenada insort	1.60	2	HNSW consulta (ef=50)	0.11	≈0
HashTable propia	1.09	1	KNN exacto fuerza bruta	1.06	1
dict (C)	1.17	1	KD-Tree 32D	0.95	≈1
LRU propia (Zipf)	0.96	1	MinHash+LSH construir	1.12	1
B+Tree insertar	1.36	≈1.05-1.1	HNSW construir	1.04	≈1.1
B+Tree bulk load	1.09	1	PageRank	1.07	1
Arreglo + insort	1.68	2	Louvain	1.22	≈1-1.2
KD-Tree 4D consulta	0.09	≈0-0.3	Label Propagation	1.22	≈1
Fuerza bruta consulta	1.01	1	Grado ponderado	0.51	0-1
Trie autocompletar	0.08	0			

Las pendientes confirman las complejidades esperadas (Tabla 3): FIFO propia 1.08 frente a 2.01 de `list.pop(0)`; Merge Sort 1.21 y heap 1.15 ($n \log n$; algo por encima del 1.1 ideal por efectos de caché y asignación de memoria en 10^7); tabla hash 1.09 ($O(1)$ por operación, 1.3–1.4 sondeos por operación con carga $\leq$0.7); B+Tree *bulk load* 1.09 (lineal) frente a 1.68 del arreglo ordenado con `insort` (cuadrático por desplazamientos). El B+Tree con inserciones aleatorias tiene pendiente 1.36 porque desde 10^6 claves (7 418 páginas) el árbol ya no cabe en el *buffer pool* de 1 024 páginas: cada inserción paga lecturas y escrituras de página reales (9.4M escrituras con 10^7 claves). Aun así, con 10^7 claves la altura es 4 y una búsqueda puntual cuesta 1.7 lecturas de disco y 23 µs; un rango de 10 000 claves, 1.7 ms. El KD-Tree 4D responde KNN exacto en 0.15–0.19 ms visitando 68–94 nodos tanto con 10^6 como con 10^7 puntos (pendiente 0.09), frente a 11 ms y 211 ms de la

fuerza bruta (1 100× en 10^7). El Trie responde autocompletado en 1–1.8 µs independientemente de n (pendiente 0.08) frente a 963 ms del escaneo lineal con 10^6 nombres. Las referencias en C son 3–15× más rápidas en tiempo constante, como se espera de Python interpretado; la forma de las curvas es la misma. *Memoria pico* con 10^7: FIFO 457 MB, heap 1.29 GB (tuplas), tabla hash 256 MB (frente a 320 MB de `dict`), KD-Tree 309 MB y B+Tree 14 MB en RAM (282 MB de páginas en disco). Límite: el Trie con 10^6 nombres ocupa 1.7 GB (4.3M nodos con su top-K cacheado), por lo que 10^7 nombres (≈17 GB) no cabe en 7.8 GB y no se ejecutó; un catálogo real de comercio electrónico rara vez supera 10^6 productos, y la alternativa sería un trie comprimido (radix) o un FST.

6.3 Filtrado probabilístico (E6)

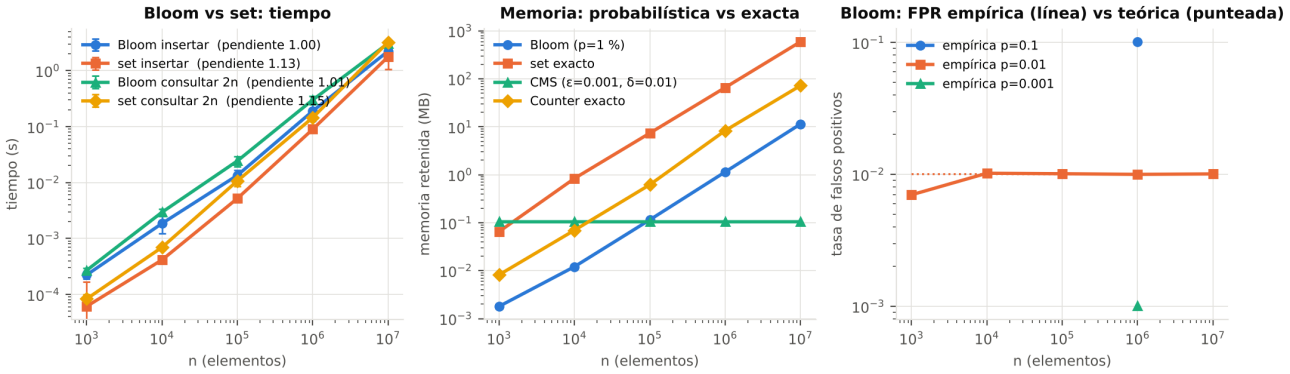


Figura 3: Bloom Filter frente a `set` (tiempo y memoria), CMS frente a `Counter` (memoria) y FPR empírica del Bloom frente a la teórica.

Cuadro 4: Estructuras probabilísticas frente a la alternativa exacta (memoria retenida medida con tracemalloc; RSS en 10^7).

Comparación	n	Construir (aprox./exacto)	Consultar $2n$	Memoria (aprox./exacto)	Calidad
Bloom (p=1 %) vs `set`	10^5	13.46 ms / 5.19 ms	24.36 ms / 10.71 ms	118 KB / 7.4 MB	FPR 1.011 % (teórica 1.004 %), FN = 0
Bloom (p=1 %) vs `set`	10^6	189.92 ms / 89.68 ms	300.52 ms / 144.64 ms	1.1 MB / 65.8 MB	FPR 1.000 % (teórica 1.004 %), FN = 0
Bloom (p=1 %) vs `set`	10^7	2.23 s / 1.77 s	3.08 s / 3.24 s	11.4 MB / 594.4 MB	FPR 1.008 % (teórica 1.004 %), FN = 0
CMS (ε=0.001) vs `Counter`	10^5	4.34 ms / 6.60 ms	–	107 KB / 635 KB	err. medio 8, máx 35 (εN=100); 100 % en cota; subestim. 0
CMS (ε=0.001) vs `Counter`	10^6	57.39 ms / 122.94 ms	–	107 KB / 8.3 MB	err. medio 138, máx 380 (εN=1000); 100 % en cota; subestim. 0
CMS (ε=0.001) vs `Counter`	10^7	654.48 ms / 2.05 s	–	107 KB / 72.3 MB	err. medio 1505, máx 40991 (εN=9997); 100 % en cota; subestim. 0

El Bloom Filter cumple la FPR teórica en todos los tamaños (1.008 % medida frente a 1.004 % teórica con 10^7 claves; 0.101 % frente a 0.100 % con $p = 0.1$ %) y nunca da falsos negativos, con 9.6 bits por elemento: **11 MB frente a 594 MB** del `set` exacto con 10^7 claves (52× menos). El tiempo es del mismo orden que el del `set` (ambos lineales; el `set` es C nativo), así que la ventaja del Bloom es de **memoria**, no de velocidad, y por eso se usa como filtro frente a la verificación exacta. El CMS ocupa 104 KB constantes ($w = 2\,719$, $d = 5$) frente a 72 MB del `Counter` con ∼950 000 claves distintas; nunca subestimó y el 100 % de las estimaciones quedó dentro de la cota εN. El precio es el error aditivo: con $N = 10^7$ el error medio es 1 505 y el error relativo en los 100 más frecuentes es 12 %; en el pipeline la ventana de 5 minutos mantiene N pequeño y el error es despreciable frente al umbral de 15.

6.4 Similitud aproximada (E7)

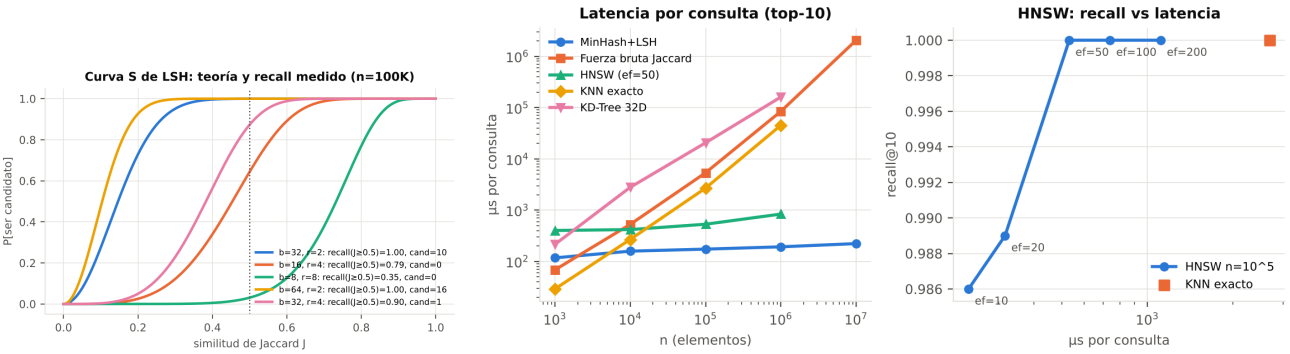


Figura 4: Izquierda: curvas S teóricas de LSH y recall medido para varias (b, r). Centro: latencia por consulta frente a n. Derecha: HNSW, recall@10 frente a latencia al variar ef ($n = 10^5$).

Cuadro 5: Técnicas aproximadas frente a fuerza bruta / KNN exacto (consultas top-10).

Técnica	n	Consulta aprox.	Exacto	Aceleración	Recall	Memoria (índice/datos)	Construcción
MinHash+LSH (32×2)	10^4	0.16 ms	0.5 ms	3×	1.00 (J≥0.5)	3.7 MB / 471 KB	117.39 ms
MinHash+LSH (32×2)	10^5	0.17 ms	5.4 ms	31×	1.00 (J≥0.5)	36.6 MB / 4.6 MB	2.77 s
MinHash+LSH (32×2)	10^6	0.19 ms	84.4 ms	438×	1.00 (J≥0.5)	366.2 MB / 45.8 MB	30.64 s
MinHash+LSH (32×2)	10^7	0.22 ms	2 057.7 ms	9 231×	1.00 (J≥0.5)	2.40 GB / 457.8 MB	232.62 s
HNSW (M=16, ef=50)	10^4	0.42 ms	0.3 ms	0.6×	1.000 (@10)	4.7 MB / 1.2 MB	29.08 s
HNSW (M=16, ef=50)	10^5	0.53 ms	2.7 ms	5.1×	1.000 (@10)	48.9 MB / 12.2 MB	296.74 s
HNSW (M=16, ef=50)	10^6	0.84 ms	45.2 ms	53.9×	0.989 (@10)	321.4 MB / 122.1 MB	3324.31 s

MinHash+LSH. La consulta cuesta ∼0.2 ms casi constante (pendiente 0.06) porque compara solo ∼10 candidatos; la fuerza bruta crece linealmente (84 ms en 10^6 y 2.1 s por consulta en 10^7): **∼9 000× más rápido en 10^7 con recall 1.0** para los vecinos con $J \geq 0.5$. El recall sobre el top-10 completo es 0.15–0.25 porque, más allá de los casi-duplicados, los vecinos tienen $J \approx 0$ y LSH está diseñado para no verlos. El costo es la memoria: el índice ocupa 2.4 GB frente a 458 MB de los conjuntos en 10^7 y se construye en 233 s. La curva S (Fig. 4) muestra el compromiso: 16×4 (umbral 0.5) pierde 21 % de los vecinos con $J \geq 0.5$; 8×8, el 65 %; 32×2 los recupera todos con 10 candidatos y 64×2 con 16. **HNSW.** Con 10^6 vectores de 32 dimensiones responde en 0.84 ms con recall@10 = 0.989, frente a 45 ms del KNN exacto (54×); con 10^5, $ef = 10$ da 0.986 en 0.24 ms y $ef \geq 50$ da recall 1.0. La construcción es el cuello de botella en Python (2.5–3.3 ms por inserción; 55 minutos para 10^6): con 10^7 serían ∼10 horas, por lo que el experimento de HNSW llega a 10^6 (en C, `hnswlib` construye 10^7 en minutos). El KD-Tree en 32D visita el 82 % de los nodos y es más lento que la fuerza bruta (161 ms frente a 45 ms): evidencia directa de la maldición de la dimensionalidad que justifica usar KD-Tree solo en 4D (E5).

6.5 Grafo: influencia y comunidades (E8)

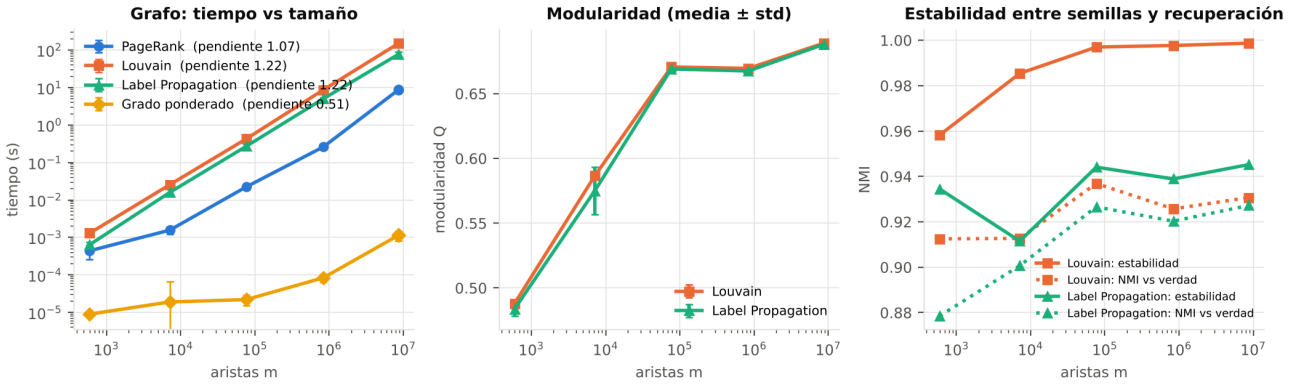


Figura 5: Grafo con comunidades plantadas (70 % de aristas internas, grados Zipf, $n/8$ nodos): tiempo, modularidad (media ± std) y estabilidad/recuperación (NMI).

Cuadro 6: Louvain frente a Label Propagation en el mismo grafo (media sobre 10 semillas; 5 en 10^6; 2 en 10^7).

Grafo (n interacciones)	Alg.	Modularidad Q	Tiempo	Nº comunidades	Mayor / mediana	Estabilidad (NMI)	NMI vs verdad
10^4 (7 146 ar.)	Louvain	0.5865 ± 0.0002	25.61 ms	17 ± 0.0	310 / 54	0.985	0.913
10^4 (7 146 ar.)	LPA	0.5750 ± 0.0183	15.96 ms	19 ± 1.4	333 / 47	0.911	0.901
10^5 (76 778 ar.)	Louvain	0.6706 ± 0.0000	429.72 ms	56 ± 0.0	783 / 184	0.997	0.937
10^5 (76 778 ar.)	LPA	0.6691 ± 0.0010	272.55 ms	85 ± 2.2	805 / 133	0.944	0.927
10^6 (835 535 ar.)	Louvain	0.6695 ± 0.0000	8.71 s	167 ± 0.6	14 491 / 431	0.998	0.926
10^6 (835 535 ar.)	LPA	0.6675 ± 0.0005	5.05 s	519 ± 12.7	15 397 / 2	0.939	0.920
10^7 (8 641 533 ar.)	Louvain	0.6889 ± 0.0000	152.04 s	553 ± 0.0	54 995 / 1360	0.999	0.931
10^7 (8 641 533 ar.)	LPA	0.6880 ± 0.0000	77.88 s	4 722 ± 47.4	58 061 / 2	0.945	0.927
Co-compra real (15 390 ar.)	Louvain	0.7553 ± 0.0018	451.55 ms	221	829 / 2	0.866	–
Co-compra real (15 390 ar.)	LPA	0.6303 ± 0.0111	190.62 ms	848	372 / 3	0.860	–

PageRank converge en 38–42 iteraciones con costo lineal en m (pendiente 1.07; 8.6 s con 8.6M aristas). En un grafo no dirigido su correlación con el grado es alta (Spearman 0.985; 95–99 % de coincidencia en el top-100), por lo que su valor diferencial está en la versión **personalizada**, que el ranking por grado no puede dar: PPR desde el carrito es la señal con mayor peso del recomendador. **Comunidades:** en el grafo sintético ambos algoritmos alcanzan modularidades parecidas, pero Louvain es **casi determinista** (NMI entre semillas 0.985–0.999 frente a 0.91–0.945; desviación de $Q \leq 2 \cdot 10^{-4}$ frente a hasta $2 \cdot 10^{-2}$ en LPA) y **no fragmenta**: LPA deja miles de comunidades diminutas (mediana 2 nodos, 4 722 comunidades en 10^7 frente a 553). LPA es ∼2× más rápido. En el grafo de co-compra *real* del pipeline (1M eventos, 5 681 productos,

15 390 aristas), la diferencia es clara: Louvain $Q = 0.755 \pm 0.002$ con 221 comunidades frente a LPA $Q = 0.630 \pm 0.011$ con 848; las comunidades de Louvain agrupan categorías complementarias plantadas (en la demo, la mayor reúne 91 laptops con 43 mouse, 42 teclados y 26 monitores).

7 Conclusiones

1. **La integración es el resultado.** Un evento atraviesa las 10 etapas y cada salida es la entrada real de otra: la lista negra exacta de E3 verifica los positivos del Bloom de E6; las alertas del CMS vuelven a E2 con prioridad 0; solo las órdenes aprobadas por E6 entran, vía *range scan* del B+Tree de E4, al grafo de E8; el PPR de E8, el LSH/HNSW de E7 y el KD-Tree de E5 se fusionan en el Top-K de E9. El pipeline procesa 10M eventos de punta a punta en Python puro.
2. **Las complejidades se cumplen empíricamente** (pendientes log-log de 1.0–1.2 para estructuras lineales y $n \log n$, 1.36 en el B+Tree por la E/S de disco, ≈0.1 para KD-Tree, Trie, LSH y HNSW, ≈2 para las alternativas ingenuas). El B+Tree muestra el efecto que justifica su existencia: con 10^7 claves, altura 4 y 1.7 lecturas de disco por búsqueda.
3. **Lo probabilístico cambia exactitud por memoria, con garantías medibles.** El Bloom cumple su FPR teórica con 52× menos memoria que un `set` (sin ganar velocidad frente a un `set` nativo); el CMS usa memoria constante, nunca subestima y respeta εN. Por eso ambos se usan como filtros frente a una verificación exacta y no como decisión final.
4. **Lo aproximado cambia recall por velocidad.** MinHash+LSH responde ∼9 000× más rápido que la fuerza bruta en 10^7 sin perder vecinos con $J \geq 0.5$, pero sus parámetros deben ajustarse al dominio (la configuración típica 16×4 fallaba con las similitudes bajas de co-compra). HNSW es 54× más rápido que el KNN exacto en 10^6 con recall 0.989; el KD-Tree, exacto y excelente en 4D, colapsa en 32D.
5. **Louvain frente a Label Propagation:** LPA es ∼2× más rápido, pero Louvain es estable entre ejecuciones, no fragmenta en miles de comunidades diminutas y, en el grafo real de co-compra, logra mayor modularidad (0.755 frente a 0.630). PageRank aporta poco sobre el grado en grafos no dirigidos; su versión personalizada es la que agrega valor.
6. **Límites y trabajo futuro:** la construcción de HNSW y el Trie de 10^7 nombres no caben en el presupuesto de 2 vCPU/7.8 GB en Python (se reportan hasta 10^6); el siguiente paso es un núcleo en C/Numba o `hnswlib`, Leiden en lugar de Louvain, *Cuckoo filters* para permitir borrados en la lista negra, y validar el pipeline con el CSV completo de REES46.

Referencias

Aumüller, M., Bernhardsson, E., & Faithfull, A. (2020). ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems, 87*, 101374. https://doi.org/10.1016/j.is.2019.02.006

Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM, 13*(7), 422–426. https://doi.org/10.1145/362686.362692

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *J. Stat. Mech.*, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems, 30*(1–7), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X

Broder, A. Z. (1997). On the resemblance and containment of documents. *Compression and Complexity of Sequences*, 21–29. IEEE. https://doi.org/10.1109/SEQUEN.1997.666900

Charikar, M. (2002). Similarity estimation techniques from rounding algorithms. *STOC '02*, 380–388. https://doi.org/10.1145/509907.509965

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: The Count-Min sketch and its applications. *Journal of Algorithms, 55*(1), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

Fan, B., Andersen, D. G., Kaminsky, M., & Mitzenmacher, M. (2014). Cuckoo filter: Practically better than Bloom. *CoNEXT '14*, 75–88. https://doi.org/10.1145/2674005.2674994

Fortunato, S., & Barthélemy, M. (2007). Resolution limit in community detection. *PNAS, 104*(1), 36–41. https://doi.org/10.1073/pnas.0605965104

Haveliwala, T. H. (2002). Topic-sensitive PageRank. *WWW '02*, 517–526. https://doi.org/10.1145/511446.511513

Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: Towards removing the curse of dimensionality. *STOC '98*, 604–613. https://doi.org/10.1145/276698.276876

Jégou, H., Douze, M., & Schmid, C. (2011). Product quantization for nearest neighbor search. *IEEE TPAMI, 33*(1), 117–128. https://doi.org/10.1109/TPAMI.2010.57

Kirsch, A., & Mitzenmacher, M. (2006). Less hashing, same performance: Building a better Bloom filter. *ESA 2006*, LNCS 4168, 456–467. https://doi.org/10.1007/11841036_42

Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of massive datasets* (3rd ed.). Cambridge University Press.

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI, 42*(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS, 103*(23), 8577–8582. https://doi.org/10.1073/pnas.0601602103

Raghavan, U. N., Albert, R., & Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E, 76*(3), 036106. https://doi.org/10.1103/PhysRevE.76.036106

REES46 (2019). eCommerce behavior data from multi category store [Dataset]. Kaggle. https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store

Samet, H. (2006). *Foundations of multidimensional and metric data structures*. Morgan Kaufmann.

Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports, 9*, 5233. https://doi.org/10.1038/s41598-019-41695-z